

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI, ERODE – 638 060

PROJECT REPORT

SMART FLOOD LEVEL MONITORING SYSTEM

USING ULTRASONIC SENSOR AND ESP8266

PROJECT TEAM

Muthu Krishnan P

Priyanth C S

Pradeep P

Sri Raman G

Smart Flood Level Monitoring System

Ultrasonic Sensor

1. Project Overview

- The project is designed to **monitor water levels continuously** using an HC-SR04 ultrasonic sensor.
- An **ESP8266 Node MCU** processes the water-level information.
- A **16×2 LCD** displays the measured water level and status.
- Three LEDs indicate **Low, Medium, and High**-water levels.
- A buzzer provides an **alert during critical flood conditions**.
- The system can be used as a prototype for **flood monitoring and disaster management**.

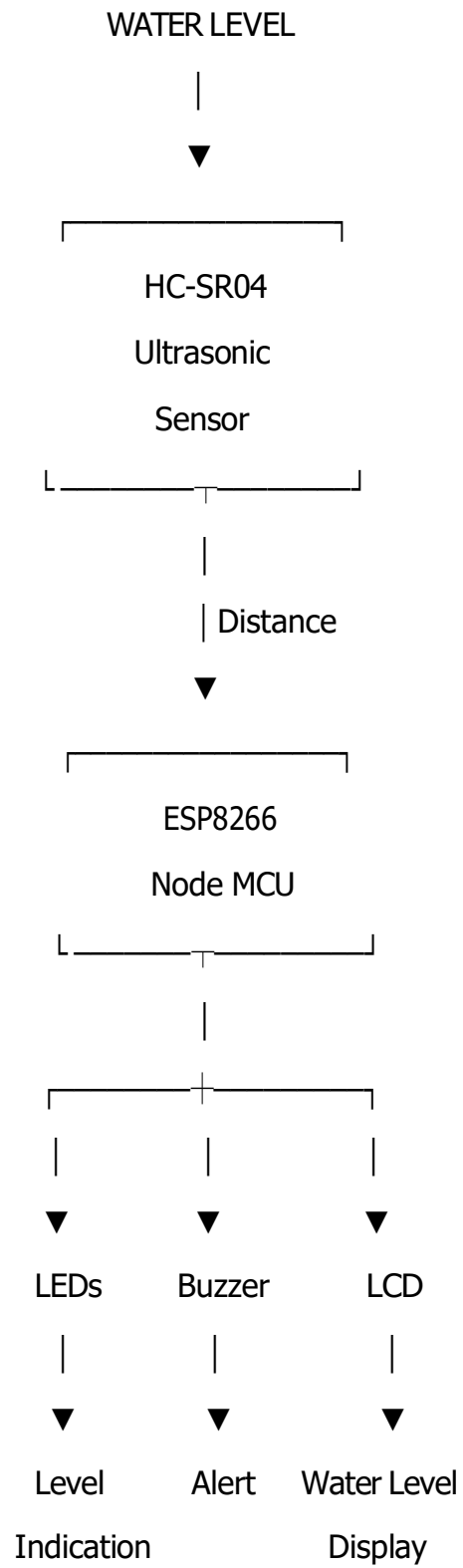
2. Problem Statement

- Floods can cause severe damage to people, buildings, agriculture, and infrastructure.
- Manual monitoring of water levels is time-consuming and unreliable.
- There is a need for a **low-cost and automatic water-level monitoring system**.
- The system should detect rising water levels continuously.
- It should provide an **early warning when the water reaches a dangerous level**.

3. Proposed Solution

- Use an **HC-SR04 ultrasonic sensor** to measure the distance between the sensor and water surface.
- Use **ESP8266** to process the sensor data.
- Display the water level on a **16×2 LCD**.
- Use LEDs to indicate different water-level conditions.
- Activate the **buzzer when the water reaches the critical level**.
- The system operates using a **5 V USB power supply**, without a battery.

4. System Architecture



5. Circuit Table

Component	ESP8266 Connection
HC-SR04 VCC	VIN / 5 V
HC-SR04 GND	GND
HC-SR04 TRIG	D5
HC-SR04 ECHO	D6 through voltage divider
Green LED	D0
Yellow LED	D3
Red LED	D4
Buzzer IO	D7
Buzzer VCC	VIN / 5 V
Buzzer GND	GND
LCD VCC	VIN / 5 V
LCD GND	GND
LCD SDA	D2
LCD SCL	D1
LED resistors	220 Ω each

Power: ESP8266 is powered through its **USB connection**. All components share a common GND.

6. Circuit Design

The HC-SR04 is positioned above the water surface. It sends ultrasonic waves toward the water and receives the reflected signal.

The ESP8266 calculates the distance using:

$$\text{Distance} = (\text{Time} \times 0.0343) / 2$$

The three LEDs are connected through **220 Ω resistors** to protect the LEDs.

The buzzer module is connected to D7 through its **IO pin**

8. Coding

```
// =====
//   SMART FLOOD LEVEL MONITORING SYSTEM
// =====
// ESP8266 + HC-SR04 + 3 LEDs + BUZZER + ADAFRUIT IO
//
// GREEN = SAFE
// YELLOW = WARNING
// RED  = FLOOD
//
// BUZZER:
// HIGH = ON
// LOW  = OFF
//
// WATER DISTANCE:
// > 150 cm    = SAFE
// 100 - 150 cm = WARNING
// < 100 cm    = FLOOD
//
// ADAFRUIT IO:
// Feed 1 = water-distance
// Feed 2 = flood-status
// =====

// =====
// LIBRARIES
// =====

#include <ESP8266WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// =====
// WIFI SETTINGS
// =====

// Put your WiFi name here
#define WIFI_SSID "Naveenraj"
```

```
// Put your WiFi password here
#define WIFI_PASSWORD "12345678"

// =====
// ADAFRUIT IO SETTINGS
// =====

#define AIO_SERVER  "io.adafruit.com"
#define AIO_SERVERPORT 1883

// Your Adafruit IO username
#define AIO_USERNAME "muthumkt"

// Generate a NEW AIO key in Adafruit IO
#define AIO_KEY      "aio_zVqu70W0tCfTybRgtOk44QdTCJjK"

// =====
// ESP8266 PIN DEFINITIONS
// =====

// HC-SR04
#define TRIG_PIN 14 // D5
#define ECHO_PIN 12 // D6

// LEDs
#define GREEN_LED 16 // D0
#define YELLOW_LED 0 // D3
#define RED_LED 2 // D4

// Buzzer
#define BUZZER_PIN 13 // D7

// =====
// FLOOD LEVEL LIMITS
// =====

const float FLOOD_LIMIT = 100.0;
```

```
const float WARNING_LIMIT = 150.0;
```

```
// =====  
// WIFI SETTINGS  
// =====
```

```
const unsigned long WIFI_TIMEOUT = 20000;
```

```
// =====  
// ADAFRUIT PUBLISH INTERVAL  
// =====
```

```
const unsigned long MQTT_PUBLISH_INTERVAL = 5000;
```

```
unsigned long lastPublishTime = 0;
```

```
// =====  
// WIFI + MQTT OBJECTS  
// =====
```

```
WiFiClient client;
```

```
Adafruit_MQTT_Client mqtt(  
  Cclient,  
  AIO_SERVER,  
  AIO_SERVERPORT,  
  AIO_USERNAME,  
  AIO_KEY  
);
```

```
// =====  
// ADAFRUIT IO FEEDS  
// =====
```

```
// Water distance feed  
Adafruit_MQTT_Publish waterDistanceFeed =  
  Adafruit_MQTT_Publish(  

```

```
Cmqtt,  
AIO_USERNAME "/feeds/water-distance"  
);
```

```
// Flood status feed  
Adafruit_MQTT_Publish floodStatusFeed =  
Adafruit_MQTT_Publish(  
  Cmqtt,  
  AIO_USERNAME "/feeds/flood-status"  
);
```

```
// =====  
// WIFI CONNECTION  
// =====
```

```
bool connectWiFi()  
{  
  Serial.println();  
  Serial.println("Connecting to WiFi...");
```

```
  WiFi.mode(WIFI_STA);
```

```
  WiFi.begin(  
    WIFI_SSID,  
    WIFI_PASSWORD  
  );
```

```
  unsigned long startTime = millis();
```

```
  while (WiFi.status() != WL_CONNECTED)  
  {  
    delay(500);
```

```
    Serial.print(".");
```

```
    // Stop trying after 20 seconds  
    if (millis() - startTime >= WIFI_TIMEOUT)  
    {  
      Serial.println();
```



```

    Serial.println("WiFi connection FAILED!");
    Serial.println();

    return false;
}
}

Serial.println();
Serial.println("WiFi connected!");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

Serial.println();

return true;
}

// =====
// ADAFRUIT IO CONNECTION
// =====

bool connectAdafruitIO()
{
    if (mqtt.connected())
    {
        return true;
    }

    Serial.println("Connecting to Adafruit IO...");

    int8_t ret = mqtt.connect();

    if (ret != 0)
    {
        Serial.print("Adafruit IO connection failed: ");
        Serial.println(mqtt.connectErrorString(ret));

        mqtt.disconnect();
    }
}

```

```

    return false;
}

Serial.println("Adafruit IO connected!");
Serial.println();

return true;
}

// =====
// HC-SR04 DISTANCE
// =====

float readDistance()
{
    unsigned long duration;

    //-----
    // Trigger ultrasonic sensor
    //-----

    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);

    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    //-----
    // Read echo
    //-----

    duration = pulseIn(
        ECHO_PIN,
        HIGH,

```

```
    30000UL  
);
```

```
//-----  
// No echo  
//-----
```

```
if (duration == 0)  
{  
    return -1;  
}
```

```
//-----  
// Calculate distance  
//-----
```

```
float distance;
```

```
distance = (duration * 0.0343) / 2.0;
```

```
return distance;  
}
```

```
// =====  
// TURN EVERYTHING OFF  
// =====
```

```
void allOff()  
{  
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(YELLOW_LED, LOW);  
    digitalWrite(RED_LED, LOW);
```

```
    // Normal buzzer:  
    // LOW = OFF  
    digitalWrite(BUZZER_PIN, LOW);  
}
```

```
// =====  
// SEND DATA TO ADAFRUIT IO  
// =====
```

```
void sendToAdafruit(  
  float distance,  
  const char* status  
)  
{  
  // Make sure WiFi is connected  
  if (WiFi.status() != WL_CONNECTED)  
  {  
    Serial.println("WiFi not connected - data not uploaded.");  
  
    return;  
  }  
}
```

```
  // Make sure MQTT is connected  
  if (!mqtt.connected())  
  {  
    if (!connectAdafruitIO())  
    {  
      return;  
    }  
  }  
}
```

```
//-----  
// Send distance  
//-----
```

```
Serial.print("Sending distance: ");  
Serial.println(distance);
```

```
if (waterDistanceFeed.publish(distance))  
{  
  Serial.println("Distance sent to Adafruit IO");  
}
```

```
else
{
  Serial.println("Failed to send distance");
}
```

```
//-----
// Send status
//-----
```

```
Serial.print("Sending status: ");
Serial.println(status);
```

```
if (floodStatusFeed.publish(status))
{
  Serial.println("Status sent to Adafruit IO");
}
else
{
  Serial.println("Failed to send status");
}
}
```

```
// =====
// SETUP
// =====
```

```
void setup()
{
  //-----
  // SERIAL MONITOR
  //-----
```

```
Serial.begin(9600);
```

```
delay(2000);
```

```
Serial.println();
Serial.println("=====");
```

```
Serial.println("  SMART FLOOD LEVEL MONITOR");  
Serial.println("=====");
```

```
Serial.println("ESP8266 STARTING...");  
Serial.println();
```

```
//-----  
// HC-SR04  
//-----
```

```
pinMode(TRIG_PIN, OUTPUT);  
pinMode(ECHO_PIN, INPUT);
```

```
digitalWrite(TRIG_PIN, LOW);
```

```
//-----  
// LEDs  
//-----
```

```
pinMode(GREEN_LED, OUTPUT);  
pinMode(YELLOW_LED, OUTPUT);  
pinMode(RED_LED, OUTPUT);
```

```
//-----  
// BUZZER  
//-----
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
//-----  
// INITIAL STATE  
//-----
```

```
allOff();
```

```
//-----
```

```

// WIFI
//-----

bool wifiConnected = connectWiFi();


//-----
// ADAFRUIT IO
//-----

if (wifiConnected)
{
    connectAdafruitIO();
}
else
{
    Serial.println("Running in LOCAL MODE.");
    Serial.println("Flood monitoring will continue.");
    Serial.println();
}


//-----
// SYSTEM READY
//-----

Serial.println("=====");
Serial.println("SYSTEM READY");
Serial.println("=====");

Serial.println("GREEN = SAFE");
Serial.println("YELLOW = WARNING");
Serial.println("RED = FLOOD");
Serial.println("BUZZER = ON below 100 cm");

Serial.println("-----");

Serial.println("TRIG = D5 / GPIO14");
Serial.println("ECHO = D6 / GPIO12");
Serial.println("GREEN = D0 / GPIO16");
Serial.println("YELLOW = D3 / GPIO0");

```

```

Serial.println("RED = D4 / GPIO2");
Serial.println("BUZZER = D7 / GPIO13");

Serial.println("-----");
Serial.println();
}

// =====
// MAIN LOOP
// =====

void loop()
{
  //-----
  // Keep MQTT connection alive
  //-----

  if (WiFi.status() == WL_CONNECTED)
  {
    if (!mqtt.connected())
    {
      connectAdafruitIO();
    }

    mqtt.processPackets(10);
    mqtt.ping();
  }

  //-----
  // READ ULTRASONIC SENSOR
  //-----

  float distance = readDistance();

  // =====
  // SENSOR ERROR
  // =====

```



```

if (distance < 0)
{
    allOff();

    Serial.println("-----");
    Serial.println("ERROR: NO ECHO FROM HC-SR04");
    Serial.println("Check ultrasonic connections:");
    Serial.println("TRIG -> D5");
    Serial.println("ECHO -> D6");
    Serial.println("-----");
    Serial.println();

    delay(1000);

    return;
}

// =====
// SERIAL DISTANCE
// =====

Serial.print("Distance: ");
Serial.print(distance, 1);
Serial.print(" cm");

// =====
// SAFE
// Distance > 150 cm
// =====

if (distance > WARNING_LIMIT)
{
    allOff();

    digitalWrite(GREEN_LED, HIGH);

    // Buzzer OFF
    digitalWrite(BUZZER_PIN, LOW);

```

```
Serial.println(" | STATUS: SAFE");
```

```
//-----  
// SEND TO ADAFRUIT IO  
//-----
```

```
if (millis() - lastPublishTime >= MQTT_PUBLISH_INTERVAL)  
{  
    lastPublishTime = millis();
```

```
    sendToAdafruit(  
        distance,  
        "SAFE"  
    );  
}  
}
```

```
//=====
```

```
// WARNING
```

```
// Distance 100-150 cm
```

```
//=====
```

```
else if (distance >= FLOOD_LIMIT)  
{  
    allOff();
```

```
    digitalWrite(YELLOW_LED, HIGH);
```

```
    // Buzzer OFF  
    digitalWrite(BUZZER_PIN, LOW);
```

```
    Serial.println(" | STATUS: WARNING");
```

```
//-----  
// SEND TO ADAFRUIT IO  
//-----
```

```
if (millis() - lastPublishTime >= MQTT_PUBLISH_INTERVAL)
```

```

{
  lastPublishTime = millis();

  sendToAdafruit(
    distance,
    "WARNING"
  );
}
}

// =====
// FLOOD
// Distance < 100 cm
// =====

else
{
  allOff();

  // Red LED ON
  digitalWrite(RED_LED, HIGH);

  // Normal Buzzer:
  // HIGH = ON
  digitalWrite(BUZZER_PIN, HIGH);

  Serial.println(" | STATUS: FLOOD ALERT!");
  Serial.println("Buzzer: ON");

  // -----
  // SEND TO ADAFRUIT IO
  // -----

  if (millis() - lastPublishTime >= MQTT_PUBLISH_INTERVAL)
  {
    lastPublishTime = millis();

    sendToAdafruit(
      distance,

```

```

        "FLOOD"
    );
}
}

//-----
// Wait
//-----

delay(500);
}

```

G. System Working

1. The system is powered through the ESP8266 USB port.
2. The HC-SR04 sends ultrasonic waves toward the water.
3. The reflected wave is detected by the sensor.
4. ESP8266 calculates the distance to the water surface.
5. The calculated value is compared with predefined thresholds.
6. **Green LED** indicates a low/safe water level.
7. **Yellow LED** indicates a medium/warning level.
8. **Red LED and buzzer** indicate a high/critical flood level.
9. The LCD continuously displays the water-level condition.
10. The process repeats continuously for real-time monitoring.

10. Prototype Implementation

- The components are assembled on a breadboard.
- The HC-SR04 is positioned above a water container.
- Different water levels are created by gradually filling the container.
- ESP8266 receives the ultrasonic measurements.
- The LCD displays the corresponding water-level condition.
- LEDs provide visual indication.
- The buzzer activates when the water reaches the critical level.

11. Bill of Materials

S No.	Component	Quantity
1	ESP8266 Node MCU	1
2	HC-SR04 Ultrasonic Sensor	1
3	16×2 I2C LCD	1
4	Green LED	1
5	Yellow LED	1
6	Red LED	1
7	220 Ω Resistor	3
8	Buzzer Module	1
9	Breadboard	1
10	Jumper Wires	As required
11	USB Cable	1
12	5 V USB Adapter	1

12. Results s Performance Analysis

Test Condition	Expected Result	System Response
Low water level	Distance >20 cm	Green LED ON
Medium water level	10–20 cm	Yellow LED ON
High water level	≤ 10 cm	Red LED + Buzzer ON
Continuous monitoring	Real-time measurement	LCD continuously updated
Critical condition	Flood warning	Audible + visual alert

Results

- The system successfully detects changes in water level using the ultrasonic sensor.
- The LCD provides real-time water-level information.
- Three LEDs provide clear visual status indications.
- The buzzer provides an immediate warning during critical conditions.
- The prototype provides a **simple, low-cost solution for flood-level monitoring and early warning.**





Devices

Feeds

Dashboards

Actions

Power-Ups

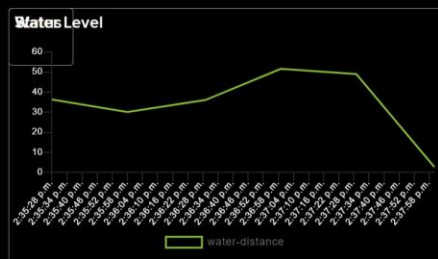


New Device

muthumkt / Dashboards / Monitor

New Block

Edit Layout

**Status**

2026/08/14 02:37:29PM Default flood-status FLOOD
2026/08/14 02:38:00PM Default flood-status FLOOD

What is Adafruit IO?

Get Help

Quick Guides

API Documentation

FAQ

Freebies

Terms of Service

Privacy Policy

Website Accessibility

Send Feedback

IO Status ●

Learn

IO Plus

News

"Ever tried. Ever failed. No matter. Try again. Fail again. Fail better"

– Samuel Beckett

```
Buzzer: ON
Distance: 4.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 4.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 5.9 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 4.2 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 5.9 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 4.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 4.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 4.6 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 3.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 3.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 3.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
```

CLOUD UPLOAD

```
Distance: 3.3 cm
Distance uploaded successfully
Status: FLOOD
Status uploaded successfully
```

```
Distance: 3.3 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 4.9 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 3.6 cm | STATUS: FLOOD ALERT!
Buzzer: ON
Distance: 3.6 cm | STATUS: FLOOD ALERT!
Buzzer: ON
```